

COP4530 Homework 2

Submission and document note

This writeup combines the four supplied section writeups into one document and keeps each explanation grounded in the matching C++ source file. `README.md` is the source document used to generate `output/pdf/COP4530_Homework2_Writeup.pdf` with `build_pdf.py`. The ZIP submission contains this README, the generated PDF, and a separate folder for each section containing its code.

The design explanations are intentionally concise. The code listings below are the complete programs submitted for the programming questions.

Section 1: Inheritance and Classes

Design decisions

The model uses `Person` as the common base class. `Student` and `Faculty` are the two main university roles, so both inherit publicly from `Person`. The more specific roles then inherit from the appropriate role:

```
Person
|-- Student
|   |-- UndergraduateStudent
|   |-- GraduateStudent
|-- Faculty
|   |-- Professor
|   |-- Instructor
```

Each class adds only data that belongs to that level. For example, every student has a major and GPA, while only a graduate student has a research area. Public inheritance expresses an "is-a" relationship: a `Professor` is a `Faculty` member and therefore also a `Person`.

Member variables

Class	Added member variables	Reason
<code>Person</code>	<code>name</code> , <code>email</code> , <code>gender</code>	Information shared by all people
<code>Student</code>	<code>major</code> , <code>minor</code> , <code>GPA</code> , <code>studentID</code>	Academic identity and progress
<code>UndergraduateStudent</code>	<code>standing</code>	Freshman through senior classification
<code>GraduateStudent</code>	<code>researchArea</code>	Graduate research focus
<code>Faculty</code>	<code>employeeID</code> , <code>department</code>	Employment information
<code>Professor</code>	<code>rank</code> , <code>researchArea</code>	Academic rank and specialty
<code>Instructor</code>	<code>coursesTaught</code>	Teaching assignment information

Complete program: Section1.cpp

```
#include <iostream>
#include <string>
using namespace std;

// Person is the common base class for everyone in the university model.
class Person {
public:
    string name;
    string email;
    string gender;
};

// A faculty member is a person who works for a department.
class Faculty : public Person {
public:
    string employeeID;
    string department;
};

// A student is a person with academic information.
class Student : public Person {
public:
    string major;
    string minor;
    float GPA;
    string studentID;
};

// An undergraduate student is a student with a class standing.
class UndergraduateStudent : public Student {
public:
    string standing;    // Freshman, Sophomore, Junior, or Senior
};

// A graduate student is a student with a research area.
class GraduateStudent : public Student {
public:
    string researchArea;
};

// A professor is faculty with an academic rank and research specialty.
class Professor : public Faculty {
public:
    string rank;        // Assistant, Associate, or Full Professor
    string researchArea;
};

// An instructor is faculty with a list of courses taught.
class Instructor : public Faculty {
public:
    string coursesTaught;
};

int main() {
    // The class declarations are the deliverable for this section. These
    // objects verify that every requested type can be instantiated.
    Person person;
    Student student;
    Faculty faculty;
    UndergraduateStudent undergraduate;
    GraduateStudent graduate;
    Professor professor;
    Instructor instructor;

    (void)person;
    (void)student;
    (void)faculty;
    (void)undergraduate;
    (void)graduate;
    (void)professor;
    (void)instructor;

    return 0;
}
```

Section 2: Dynamic Casting

Output

```
Read it.  
Ship it.  
Buy it.  
Read it.  
Box it.  
Read it.
```

Reasoning

`printMe()` is virtual, so C++ selects the override belonging to the object's actual type. A pointer's declared type does not replace the object's type, and `dynamic_cast` only changes the pointer view. Every cast in this example succeeds because it asks for either the object's actual type or one of its public base types.

1. `md` points to a `Maryland`, so the first call prints `Read it.`
2. `mid` points to a `State`, so the second call prints `Ship it.`
3. `obj` initially points to a `Place`, so the third call prints `Buy it.`
4. After `obj = md`, the down-cast reaches the `Maryland` object and prints `Read it.`
5. After `obj = usa`, the object is a `Region`; viewing it as a `Place` still dispatches to `Region::printMe()`, so it prints `Box it.`
6. After `usa = md`, the pointer refers to a `Maryland`, so the last call prints `Read it.`

The `Object` destructor is virtual so that the hierarchy remains safe for polymorphic ownership. Smart pointers also ensure that the four demonstration objects are released automatically.

Complete program: Section2.cpp

```
#include <iostream>
#include <memory>
using namespace std;

class Object {
public:
    virtual ~Object() = default;
    virtual void printMe() = 0;    // abstract: Object cannot be instantiated
};

class Place : public Object {
public:
    void printMe() override { cout << "Buy it.\n"; }
};

class Region : public Place {
public:
    void printMe() override { cout << "Box it.\n"; }
};

class State : public Region {
public:
    void printMe() override { cout << "Ship it.\n"; }
};

class Maryland : public State {
public:
    void printMe() override { cout << "Read it.\n"; }
};

int main() {
    // Smart pointers own the objects, so the example has no memory leaks.
    auto state = make_unique<State>();
    auto maryland = make_unique<Maryland>();
    auto place = make_unique<Place>();
    auto region = make_unique<Region>();

    Region* mid = state.get();           // object is State
    State* md = maryland.get();          // object is Maryland
    Object* obj = place.get();           // object is Place
    Place* usa = region.get();           // object is Region

    md->printMe();                       // Maryland -> Read it.
    mid->printMe();                       // State -> Ship it.
    (dynamic_cast<Place*>(obj))->printMe(); // Place -> Buy it.

    obj = md;                           // obj now points to Maryland
    (dynamic_cast<Maryland*>(obj))->printMe(); // -> Read it.

    obj = usa;                           // obj now points to Region
    (dynamic_cast<Place*>(obj))->printMe(); // Region -> Box it.

    usa = md;                           // usa now points to Maryland
    (dynamic_cast<Place*>(usa))->printMe(); // -> Read it.

    return 0;
}
```

Section 3: Generic Types

Design decisions

Pair is a class template with two independent type parameters, A and B. The compiler creates a type for each combination used in main, so the same class can store an integer and string, a float and long, or any other pair of streamable types. The constructor initializer list initializes both members directly, and printMe() is const because printing does not change the pair.

Output

```
1, apple
3.14, 1000000
A, 2.718
active, 1
65, Z
```

The fourth line contains 1 because cout prints a bool as 1 or 0 by default. boolalpha could be enabled if true and false were preferred.

Complete program: Section3.cpp

```
#include <iostream>
#include <string>

using namespace std;

template <typename A, typename B>
class Pair {
private:
    A first;
    B second;

public:
    Pair(const A& firstValue, const B& secondValue)
        : first(firstValue), second(secondValue) {}

    void printMe() const {
        cout << first << ", " << second << '\n';
    }
};

int main() {
    Pair<int, string> p1(1, "apple");
    Pair<float, long> p2(3.14f, 100000L);
    Pair<char, double> p3('A', 2.718);
    Pair<string, bool> p4("active", true);
    Pair<int, char> p5(65, 'Z');

    p1.printMe();
    p2.printMe();
    p3.printMe();
    p4.printMe();
    p5.printMe();

    return 0;
}
```

Section 4: Linked List Programming

Recursive algorithm

```
reverseRecursive(head):
    if head is null or head.next is null:
        return head

    newHead = reverseRecursive(head.next)
    head.next.next = head
    head.next = null
    return newHead
```

System design and correctness

The base case handles an empty list or a one-node list because neither needs a link changed. Otherwise, the function first reverses the suffix beginning at head->next. When the recursive call returns, that successor is at the end of the reversed suffix. Setting head->next->next = head points it back to the current node, and setting head->next = nullptr makes the current node the new tail. The same newHead is returned through every stack frame, so the original last node becomes the final head.

Each node is visited once, giving $O(n)$ time. The recursion uses $O(n)$ stack space. The program also deletes the nodes after printing to avoid a memory leak.

Output

```
Original: 1 2 3 4 5
Reversed: 5 4 3 2 1
```

Complete program: Section4.cpp

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;

    Node(int value) : data(value), next(nullptr) {}
};

// Recursively reverse the list and return its new head.
Node* reverseRecursive(Node* head) {
    if (head == nullptr || head->next == nullptr) // base case
        return head;

    Node* newHead = reverseRecursive(head->next); // reverse the rest

    head->next->next = head; // successor now points back to head
    head->next = nullptr; // head becomes the tail

    return newHead; // same new head bubbles up
}

void printList(Node* head) {
    for (Node* cur = head; cur != nullptr; cur = cur->next)
        cout << cur->data << " ";
    cout << "\n";
}

// Release every node after the demonstration is complete.
void deleteList(Node* head) {
    while (head != nullptr) {
        Node* next = head->next;
        delete head;
        head = next;
    }
}

int main() {
    // Build: 1 -> 2 -> 3 -> 4 -> 5
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);

    cout << "Original: ";
    printList(head);

    head = reverseRecursive(head);

    cout << "Reversed: ";
    printList(head);

    deleteList(head);
    return 0;
}
```

Build and run

Each section is a standalone C++17 program:

```
g++ -std=c++17 -Wall -Wextra -pedantic Section1.cpp -o Section1
g++ -std=c++17 -Wall -Wextra -pedantic Section2.cpp -o Section2
g++ -std=c++17 -Wall -Wextra -pedantic Section3.cpp -o Section3
g++ -std=c++17 -Wall -Wextra -pedantic Section4.cpp -o Section4
```

The ZIP package mirrors the requested submission structure:

```
COP4530_Homework2_Submission.zip
|-- README.md
|-- COP4530_Homework2_Writeup.pdf
|-- Section1/Section1.cpp
|-- Section2/Section2.cpp
|-- Section3/Section3.cpp
`-- Section4/Section4.cpp
```